Kafka 的技術名詞講解與概覽

用一筆訂單事件，理解 Kafka 的運作

`01001` 訂單成立之後，資料去哪裡？

課程目標

1. Kafka 是什麼？系統為什麼會需要它？
2. Producer、Consumer、Broker 分別在做什麼？
3. Topic、Partition、Key、Offset 之間是什麼關係？
4. Consumer Group 如何分工？
5. Kafka 如何保存資料、處理故障？
6. 一筆 Event 從產生到被處理，中間發生了什麼？

一、Kafka 是什麼？為什麼需要 Kafka？

先拆開「分散式」與「事件串流」，再看下單流程為什麼需要 Kafka。

Kafka 是什麼？

Kafka 是開源的分散式事件串流平台

用來擷取、儲存與處理持續產生的資料

分散式系統
多台電腦透過網路協作

事件串流
持續產生、保存與處理紀錄

這是兩個面向：系統如何協作，以及資料如何流動

Distributed System：分散式系統

多台電腦協作	透過網路，共同完成系統的工作
資料分散保存	不必把所有資料放在同一台伺服器
其中一台故障	透過備份資料與故障切換，盡量維持服務

後面會遇到：Replication、Leader / Follower、Failure、Retry

Event Streaming：事件串流

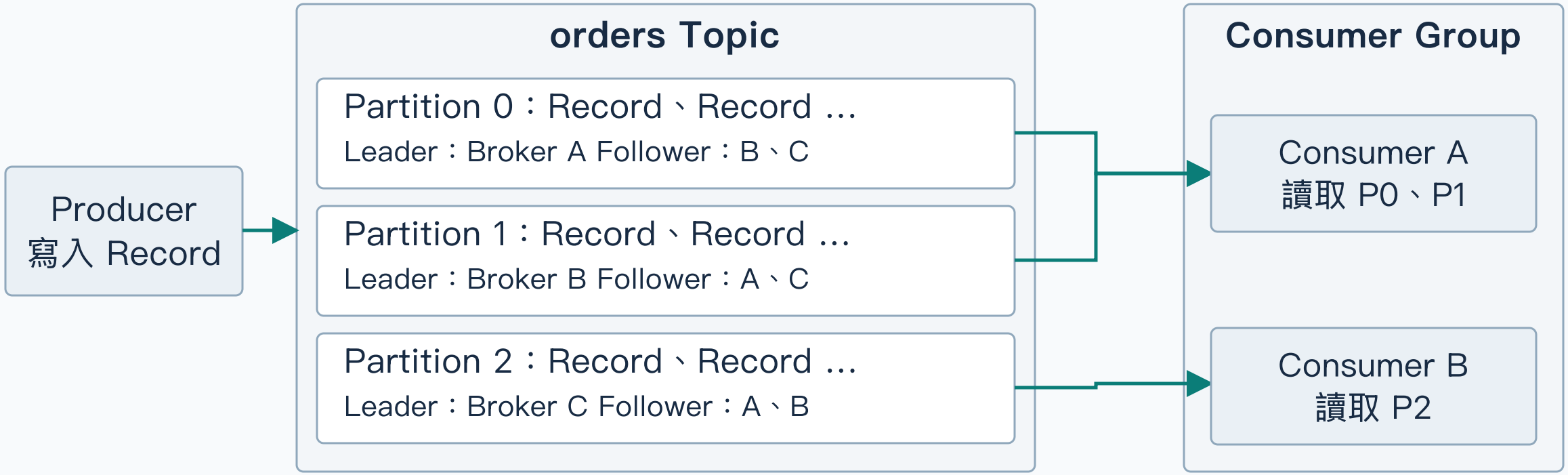
類型	訊息	意義
Command	`PlaceOrder`	請建立訂單
Event	`OrderPlaced`	訂單已成立



Event 是對已發生事情的紀錄

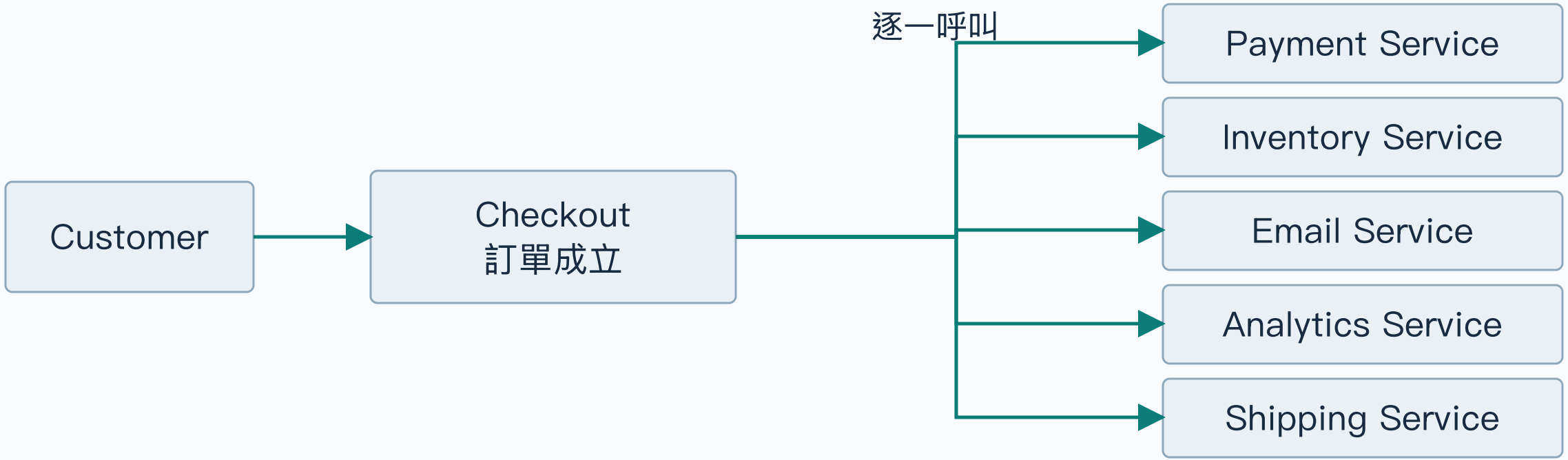
事件持續產生，就形成 Event Stream

先看整張圖



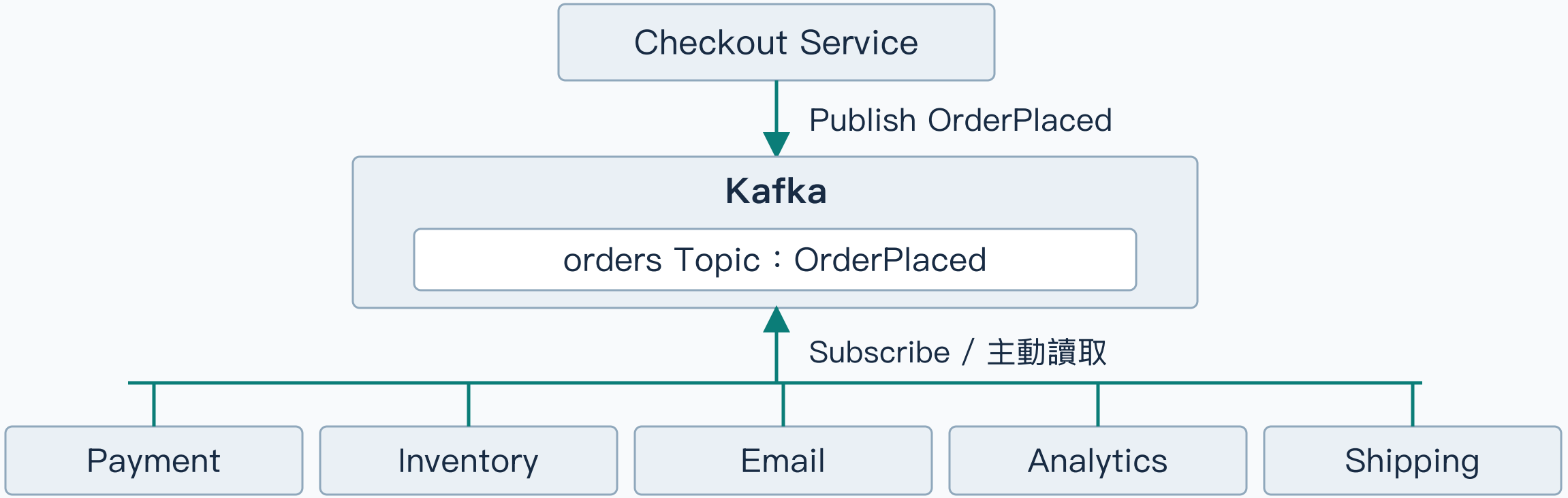
Topic 包含多個 Partition，Group 內的 Consumer 按 Partition 分工

用下單流程理解 Kafka 的用途



下游很慢或故障時，Checkout 必須決定等待、重試或回報錯誤

若使用 Kafka



Email 暫停時，其他服務仍可讀取；Email 恢復後可繼續讀取尚未清除的事件

Event 處理大致流程

1. Publish	Checkout 發布一筆 OrderPlaced Event
↓	
2. Store	Kafka 把 Event append 到 Topic 中並保存
↓	
3. Subscribe	Payment、Inventory、Email、Analytics、Shipping 訂閱需要的 Event
↓	
4. Process	每個 Service 按自己的進度讀取並處理

二、一筆 Event 在 Kafka 內的旅程

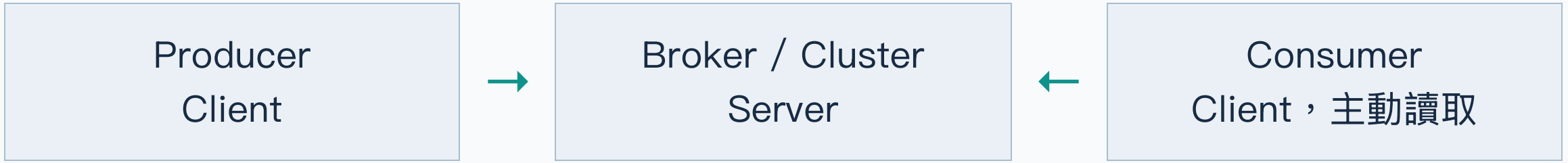
訂單 O1001 已成立，現在追蹤 OrderPlaced 的寫入與讀取。

一筆 Event 在 Kafka 內的旅程



依箭頭從上排左側開始，再往下排由右往左走完事件旅程

Client 與 Server 的分工



角色	責任
Producer	寫入 Record
Broker	保存與提供 Partition 資料
Consumer	主動讀取，交給應用程式處理

Producer 與 Serialization



Producer 是把 Record 寫入 Kafka 的 Client

Serialization 將程式資料轉成可傳輸的 bytes

Record / Event：資料結構與業務意義

```
寫入位置：orders Topic
Record
├─ Key: "O1001"
├─ Value: {
│    "eventId": "E9001",
│    "eventType": "OrderPlaced",
│    "orderId": "O1001",
│    "amount": 1200
│  }
├─ Timestamp
└─ Headers:
   └─ event-type: "OrderPlaced"
```

Event：描述已發生的事情
例如「O1001 訂單已成立」

Record：Kafka 傳輸與保存的一筆資料
這筆 Event 可表示為左側 Record

Key 影響分區，**Value** 是主要資料
Timestamp 是時間，**Headers** 放額外資訊

Broker 與 Cluster

Broker 是 Kafka 叢集中負責資料讀寫與保存的伺服器角色

Kafka Cluster

Broker A	Broker B	Broker C
orders / P0：Leader	orders / P0：Follower	orders / P0：Follower
orders / P1：Follower	orders / P1：Leader	orders / P1：Follower
orders / P2：Follower	orders / P2：Follower	orders / P2：Leader

多個 Broker 組成 Cluster；Partition 可分散到不同 Broker，同一 Partition 的 Replica 也跨 Broker 保存

Topic：資料的邏輯分類

Topic 名稱	可能承載的 Event（示例）
`orders`	OrderPlaced、OrderCancelled
`payments`	PaymentCompleted、PaymentFailed
`shipments`	OrderShipped、ShipmentDelivered



Consumer 與 Deserialization

Consumer 是從 Kafka 讀取資料的 Client，Java 常透過 `poll(...)` 取得資料

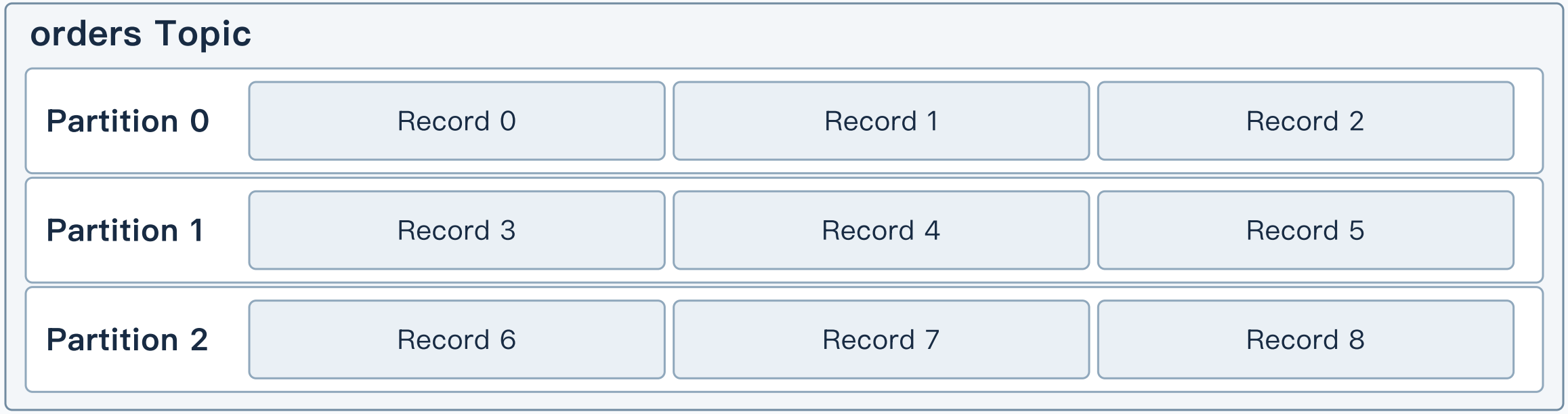


Producer 與 Consumer 必須約定資料格式；接下來看 Topic 裡的分區與順序

三、Topic、Partition、Key 與 Offset

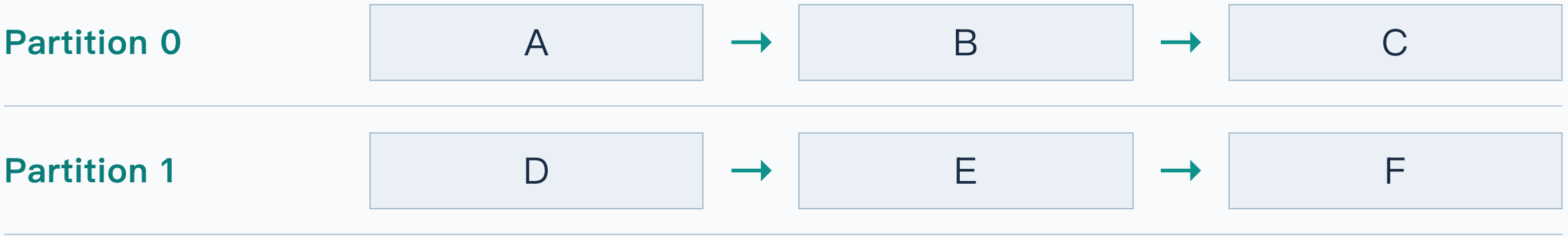
資料放在哪裡？同一張訂單的順序與讀取位置如何表示？

Topic、Partition、Record 的層級



- Topic 是邏輯分類；Partition 是 Topic 裡的有序 Log
- 每筆 Record 只寫入一個 Partition，分區讓資料分散保存與平行讀取

順序保證的範圍



可以依序讀取 A、B、C，無法推論 A 與 D 的先後

平行工作執行緒也可能改變業務完成順序

Key：讓同一張訂單進入同一分區



前提：預設 Key 分區方式，Topic 分區數量與分區規則不變

Consumer 可依同一 Partition 的寫入順序，讀取 O1001 的三筆事件

Key 的順序保證與使用條件

寫入方式	可以依賴的範圍
同一 Topic、相同 Key，分區數與規則不變	進入同一 Partition，依其寫入順序讀取
分別寫到 orders、payments、shipments	不提供跨 Topic 順序保證

指定 Partition、自訂分區規則，或增加 Partition 數量，都可能改變相同 Key 的分區結果

Offset：Partition 內的位置



不同 Partition 可各自有 Offset 0：`orders / P0 / 0` 與 `orders / P1 / 0` 是不同位置

清理後只剩 Offset **1、3**，不會重新編成 0、1；Offset 不是訂單 ID 或 Event ID

Partition 是一條 Log



讀取與 Commit 都不會取走 Record；新 Record 持續附加在 Log 尾端

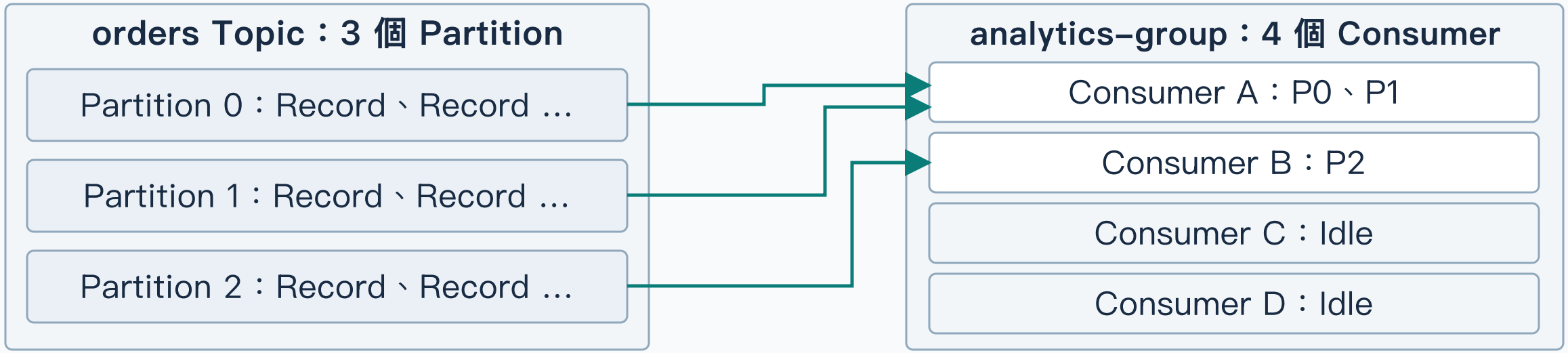
何時清理？Retention 依時間 / 容量清理舊資料，Compaction 依 Key 清理較舊值

四、Consumer Group：多個 Consumer 怎麼分工？

有多個 Partition 與多個 Consumer，怎麼分配工作與記錄進度？

Consumer Group 的 Partition 分工

Consumer Group 是一群共同分工讀取 Topic 的 Consumer



同一 Group 中，Partition 同時只給一個 Consumer；一個 Consumer 可負責多個 Partition

3 個 Partition + 4 個 Consumer：最多只有 3 個 Consumer 可以同時負責 Partition

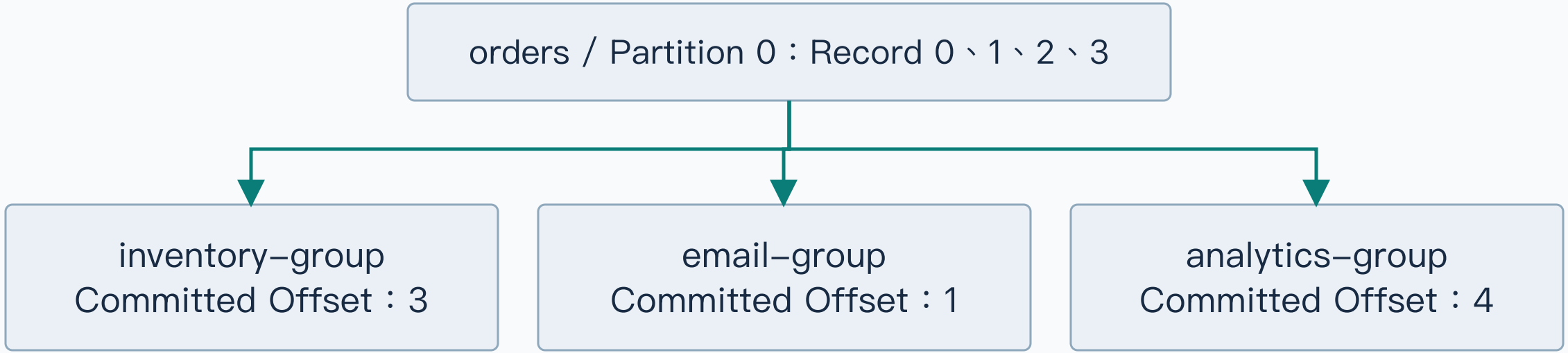
Partition 數量限制消費的平行處理能力

Partition 數量	Consumer 數量	最多 Active Consumer
1	4	1
3	4	3
6	4	4
6	8	6

Partition 決定平行讀取上限，Consumer 決定實際投入多少處理資源

分區數要考慮 Throughput、Ordering 與資源成本，更多 Partition 不代表一定更快

不同 Group，各自讀取同一份資料

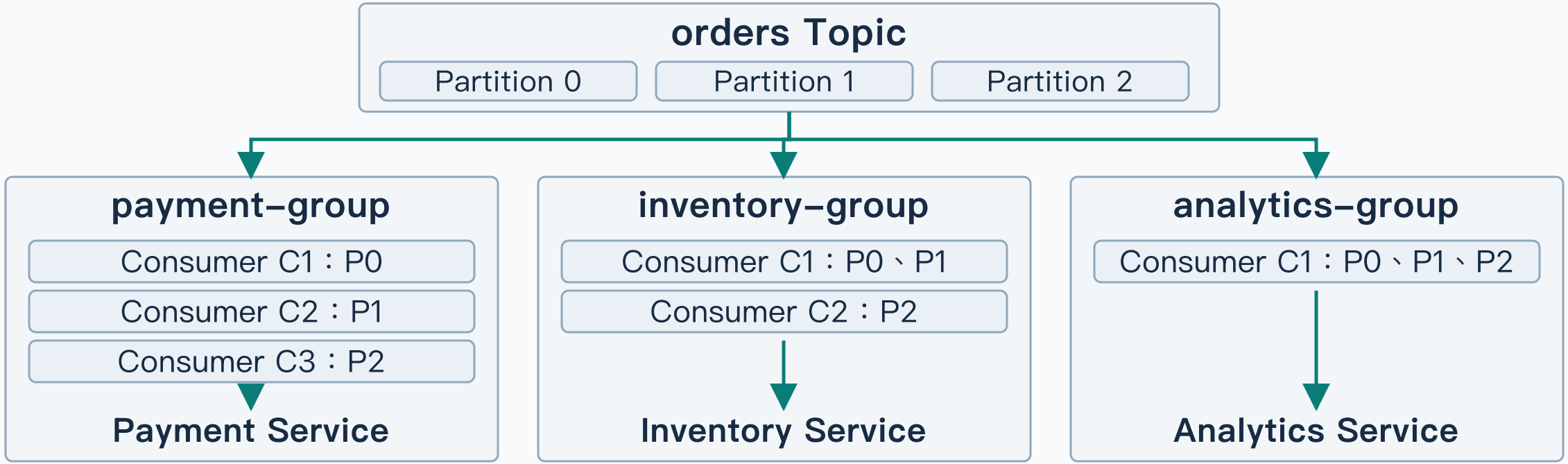


不同 Group 可獨立消費同一份資料，各自維護消費進度

一個 Group 的消費，不會改變其他 Group 的進度

Group 對應獨立的消費需求，Consumer 數量取決於處理能力與平行度需求

Consumer Group、Consumer 與 Downstream Service



- Group 數量：需要獨立消費事件的應用或服務
- Consumer 數量：流量、處理能力與部署規模
- Partition 數量：決定單一 Consumer Group 對此 Topic 的最大消費平行度。

Committed Offset 指向下一筆

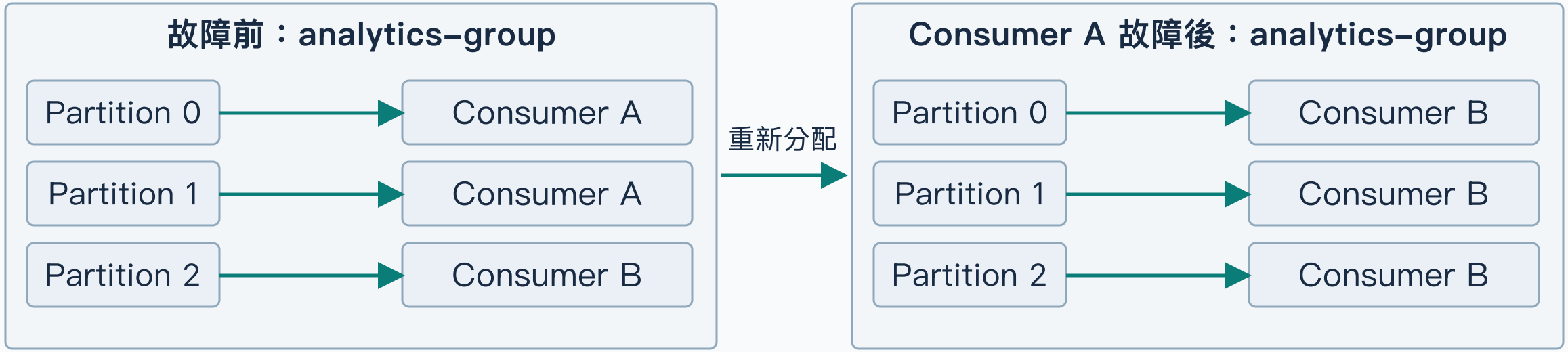


Commit 5 表示**從 Offset 5 恢復消費**，不表示 Offset 5 已處理完成

Committed Offset 是 Group 針對特定 **Topic / Partition** 記錄的進度

Commit 只記錄消費進度，不代表業務處理成功。

Rebalance：重新分配責任



改變的是誰負責消費，不是 Record 在 Partition 中的儲存順序。

- Consumer 加入、離開、故障，或 Topic Partition 數量變更，都可能觸發 Rebalance
- Rebalance 可能造成 Record 重複處理，例如 Consumer A 已經處理 Offset 2，卻還沒 Commit，Consumer B 接手後可能重新讀到 Offset 2。

Consumer Lag：進度與尾端的距離



Lag 反映 Group 的已提交進度距離 Partition 尾端有多遠

- 持續上升，可能代表消費速度跟不上資料產生速度
- 不能直接指出原因，也不一定等於尚未完成的業務筆數

Replay：重新讀取保留中的資料

昨天的 Analytics 有 Bug → 修正程式 → 將讀取位置移回舊 Offset → 重新計算

修正 Analytics 程式後，重讀 Kafka 仍保留的歷史 Records 並重新計算

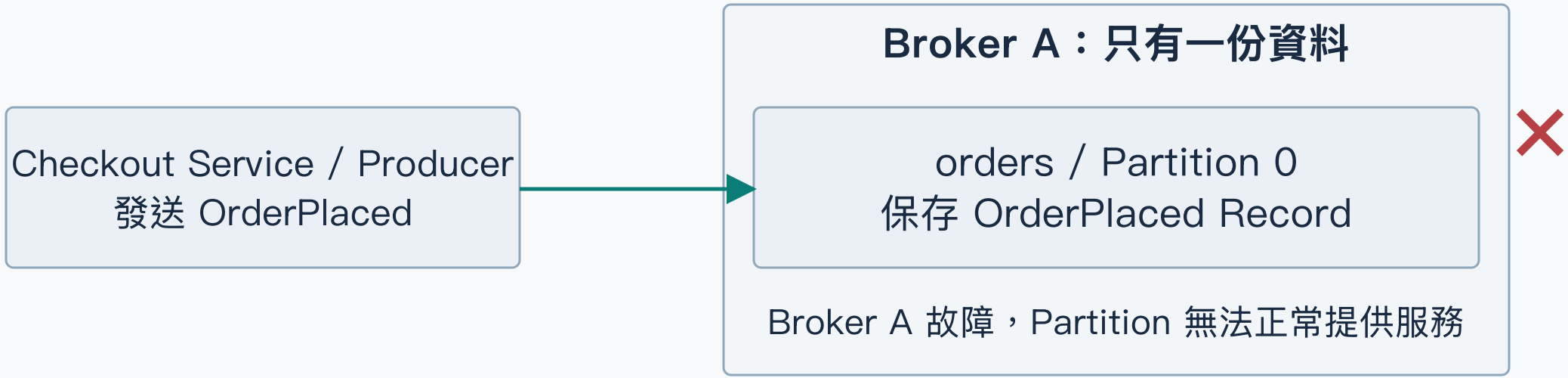
Replay 也可用於重新計算新統計，**不只用於故障恢復**

可能再次觸發寄信、扣款等副作用，必須留意重複處理

五、Kafka 怎麼保存資料、面對故障？

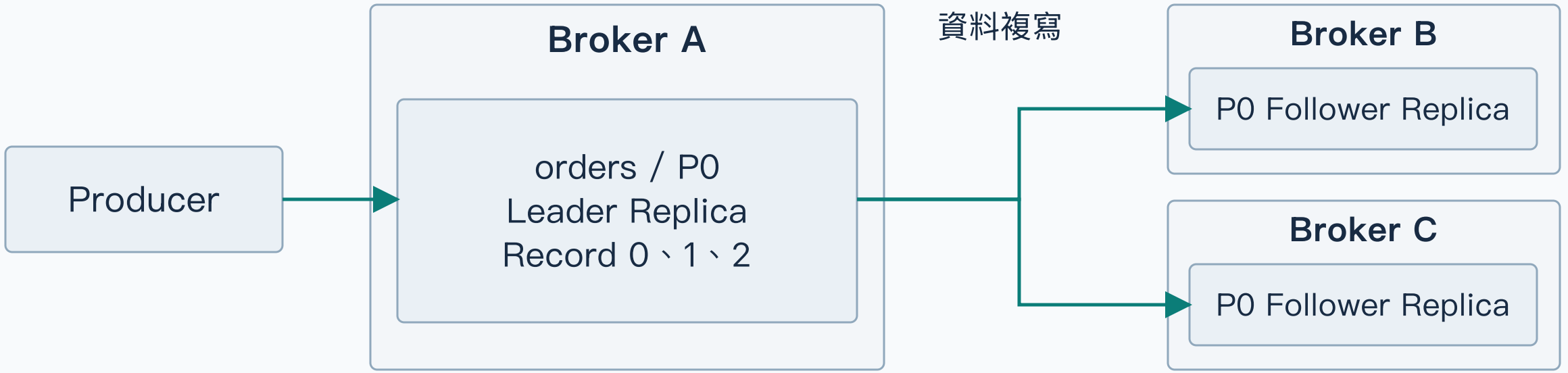
1. Broker 故障時，Kafka 如何避免資料遺失？
2. Records 持續增加，Kafka 如何管理儲存空間？

單一 Broker 有什麼風險？



如果把同一 Partition 複製到多個 Broker，能否降低單點故障的風險？

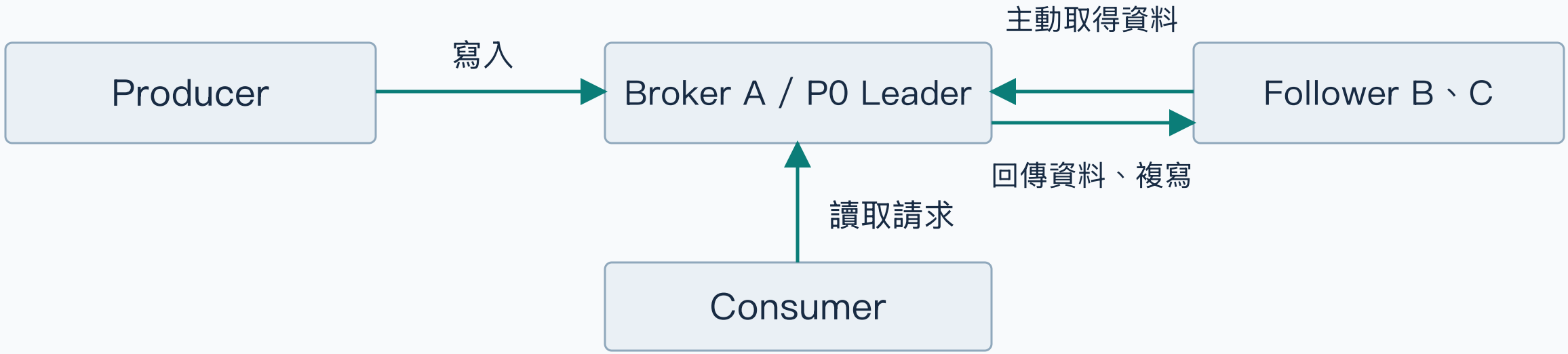
Replication：一個 Partition 的三份副本



Replication Factor = 3：同一 Partition 有三份 Replica，包含 Leader

Replication 讓 Partition 不必依賴單一 Broker 保存資料

Leader / Follower：誰處理讀寫？



Producer 向 Partition Leader 寫入；Follower 主動從 Leader 取得資料並複寫

Consumer 一般從 Leader 讀取；Leader 故障時，需要選出新 Leader

ISR：哪些 Replica 有資格接手？

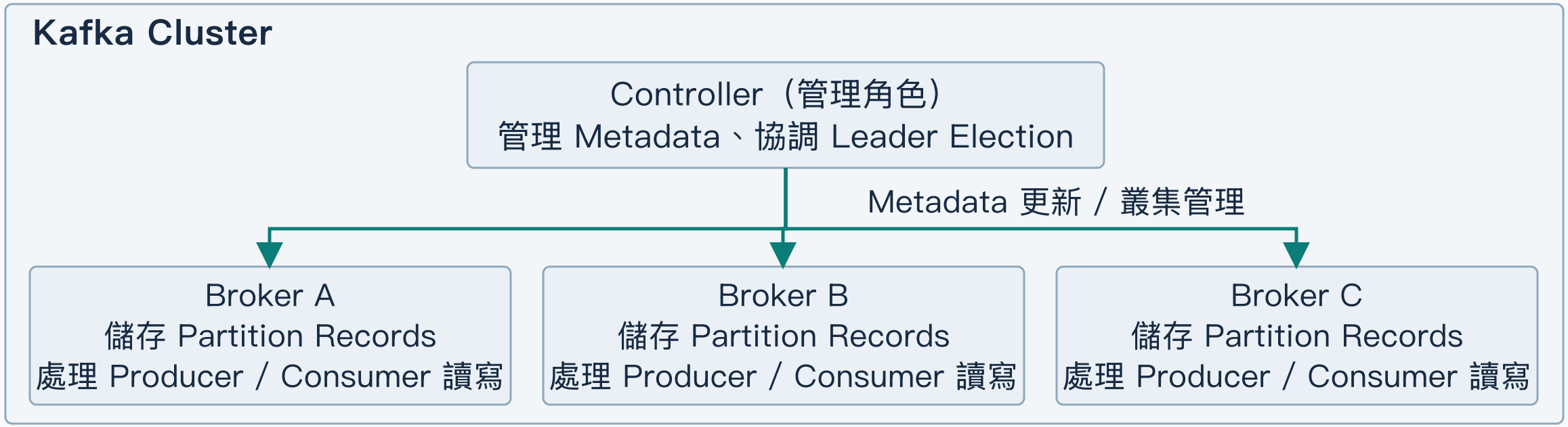
ISR（In-Sync Replicas）是符合 Kafka 同步條件的 Replica 集合

Broker	P0 Replica 狀態	ISR
A	Leader	✓
B	Follower，持續複寫並符合同步條件	✓
C	Follower，落後太久	×

Follower 不必每一瞬間完全相同，但要持續複寫、在允許時間內跟上

ISR = [A, B]：Leader 故障時，通常從合格 ISR 選出新 Leader

Controller：誰管理 Kafka Cluster？



Cluster Metadata：Topic、Partition、Replica、Leader 等叢集管理資訊

Broker 與 Controller 是 Kafka Server 的兩種角色，可同節點兼任，也可分開部署

KRaft：Controller 自己故障怎麼辦？

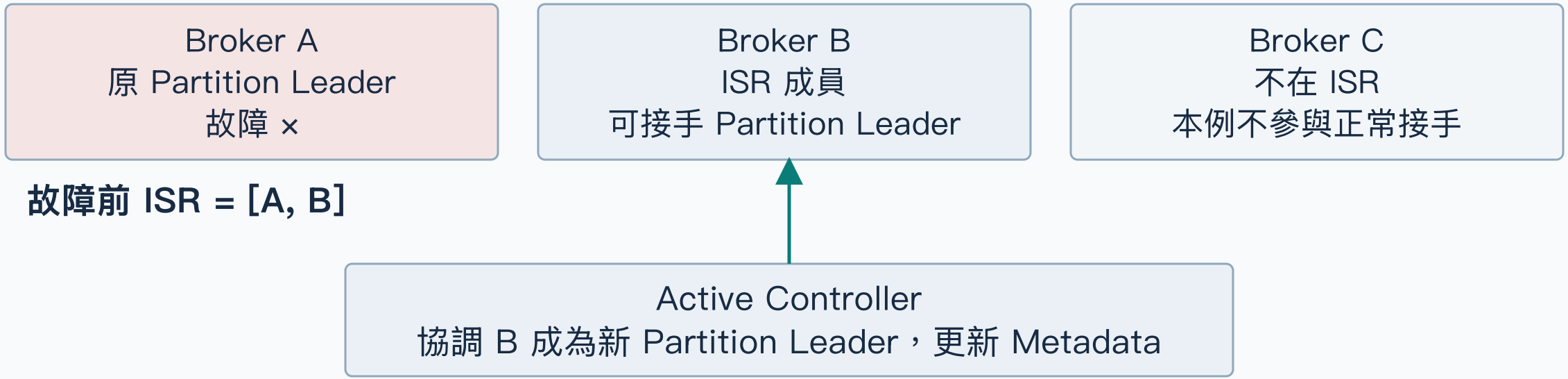
如果負責管理 Cluster 的 Controller 自己故障了呢？



Active Controller 負責目前的管理決策，其他 Controller 可參與選舉與接手

KRaft 複製 Metadata，並在 Active Controller 故障後選出接手者

Broker 故障後，誰決定新 Leader？



Active Controller 負責叢集管理；Partition Leader 負責特定 Partition 的資料讀寫

Kafka 如何決定保留哪些資料？

`cleanup.policy` 決定 Topic 使用哪種資料清理策略



`delete,compact` 可同時啟用：按 Key 整理較新狀態，仍受時間 / 容量清理限制

Retention：資料保留與清理

先看 `delete` 分支：orders 的歷史資料要保留多久、占用多少空間？

`cleanup.policy=delete` `retention.ms=604800000`（七天）



符合條件的資料以 Segment 為單位背景清理，並非到期逐筆立即刪除

Retention 依時間或容量清理；Replay 的前提是資料仍存在

Segment：為什麼 Partition Log 需要分段？

Retention 決定何時清理；Kafka 如何有效管理、刪除很大的舊 Log？



- Kafka 自動建立與輪替 Segment，以 `segment.bytes`、`segment.ms` 控制條件
- 分段方便管理、索引與清理；Retention 通常刪除符合條件的舊 Segment

Segment 是管理 Partition Log 的基本單位之一

Log Compaction：需要完整歷史，還是最新狀態？

需求 A：完整訂單事件歷史

OrderPlaced
PaymentCompleted
OrderShipped

需求 B：訂單目前的狀態

O1001 → CREATED
O1001 → PAID
O1001 → SHIPPED

`delete` 分支：在保留期限內保存事件歷史，才能回顧與重新計算

接著看 `compact` 分支：重建每個 Key 的較新狀態，不必保留所有舊版本

Log Compaction：依 Key 保留較新狀態

背景依 Key 清理舊版本，保留較新值；Offset 不重編號，清理不會立即完成

Compaction 前

Offset 0 O1001 → CREATED	Offset 1 O1002 → CREATED	Offset 2 O1001 → PAID	Offset 3 O1001 → SHIPPED

↓ Background Compaction / 背景清理

Compaction 後（示意）

Offset 0 已清理	Offset 1 O1002 → CREATED	Offset 2 已清理	Offset 3 O1001 → SHIPPED

需要保存的資料	cleanup.policy
保留期限內的完整事件歷史	`delete`
重建每個 Key 的最新狀態	`compact`
時間 / 容量限制與最新狀態整理	`delete,compact`

六、網路失敗、Retry 與可靠性

可靠性要分清三層責任：

1. **Producer**：寫入 Kafka，等待確認與避免重試造成重複 Record
2. **Consumer**：讀取 Record，提交與恢復消費進度
3. **應用程式 / 寄信服務**：決定寄信是否成功、如何避免重複寄送

Timeout：這筆 OrderPlaced 有寫入嗎？



Producer 只看到 Timeout，無法判斷 Broker 沒收到，還是回覆丟失

Retry：再送一次會不會多一筆？



重試可能讓未成功的請求完成，也可能重複寫入已成功的事件

重試與避免重複，需要一起考慮

acks：Broker 何時回覆寫入確認？

Acknowledgement 是寫入確認，Producer 透過 `acks` 指定等待條件

設定	這筆 OrderPlaced 等到什麼程度？
`acks=0`	不等待 Broker 確認
`acks=1`	Leader 寫入自己的 Log 就回覆
`acks=all`	等待目前 ISR 達成確認條件

all 指目前 ISR，不等於最初配置的所有 Replica

如果 ISR 只剩 A 一份，`acks=all` 是否仍可成功？

min.insync.replicas：只剩一份時還接受寫入嗎？

示例：`Replication Factor=3`，Producer 使用 `acks=all`

`min.insync.replicas=2`：成功寫入至少要有兩份符合條件的 ISR

當前 ISR	寫入結果
A、B、C	可接受，等待目前 ISR 確認
A、B	可接受，等待目前 ISR 確認
A	拒絕寫入

用最低同步副本要求，避免只剩一份時仍確認成功

Idempotence：重試如何避免重複寫入？

Producer Idempotence 是 Kafka 內建的功能，現代 Java Producer 預設啟用

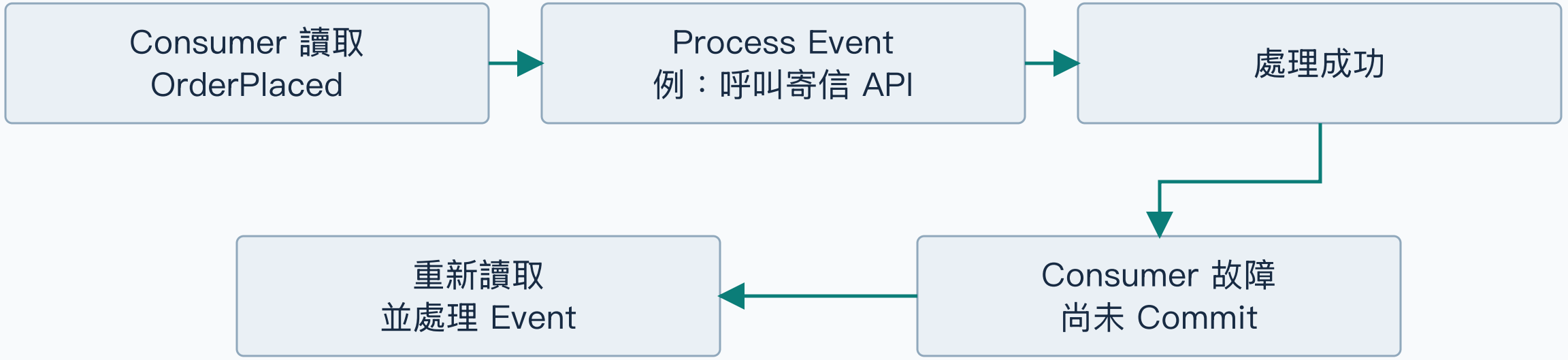
`enable.idempotence=true`，前提是沒有衝突設定

同一次發送
因 Timeout 重試
→
Producer Idempotence
識別重試
→
Kafka Log
避免新增重複 Record

去重層次	責任與範圍
Kafka Log 的寫入去重	Producer Idempotence 處理涵蓋的發送重試
寄信等業務操作的去重	應用程式 / 外部 API 的冪等與去重設計

Log 只有一筆 Record，Consumer 重讀仍可能再寄一次信

Event Processing：處理成功，但 Commit 前故障



同一筆 Event 可重新處理；Producer Idempotence 無法防止重複業務操作

Event Processing 與 Offset Commit：順序有什麼影響？

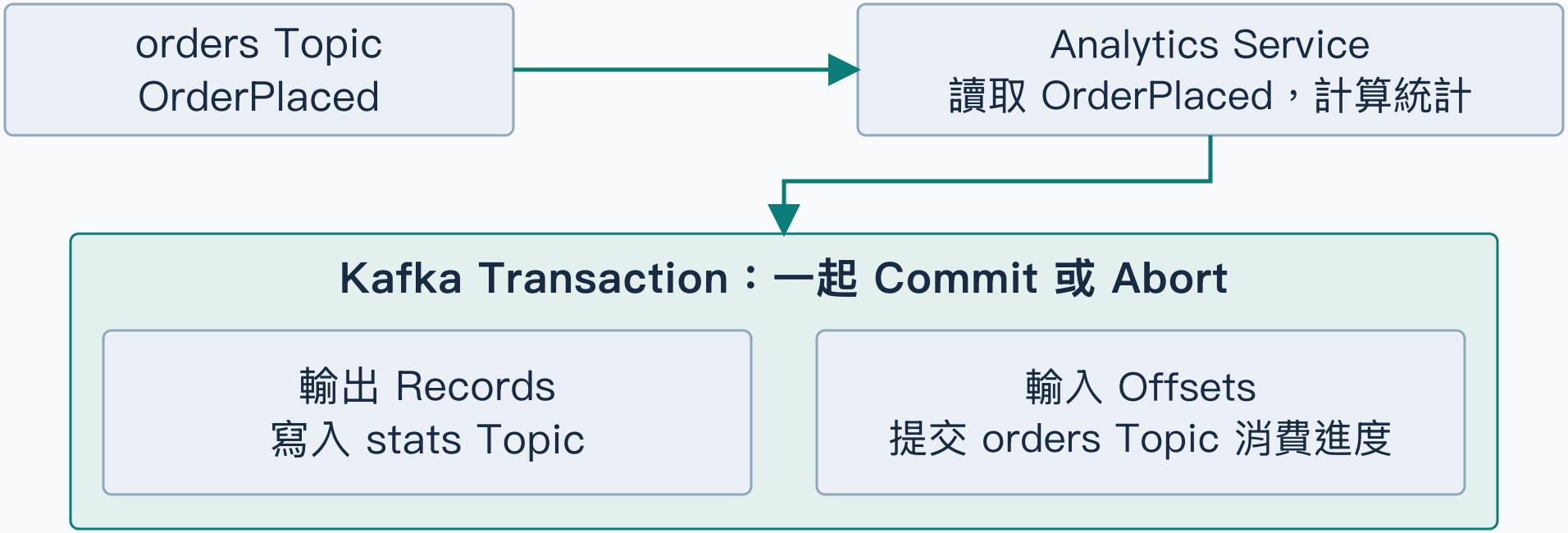
Process Event 是應用程式執行業務邏輯，不等於 Consumer 只是讀到 Event



例如寄信：先 Commit 可能漏寄；先 Process 再 Commit，可能重複呼叫寄信 API

Exactly-once：Kafka-to-Kafka 的處理保證

如果 Consumer 處理後還要寫回 Kafka，如何避免故障重試造成重複輸出？



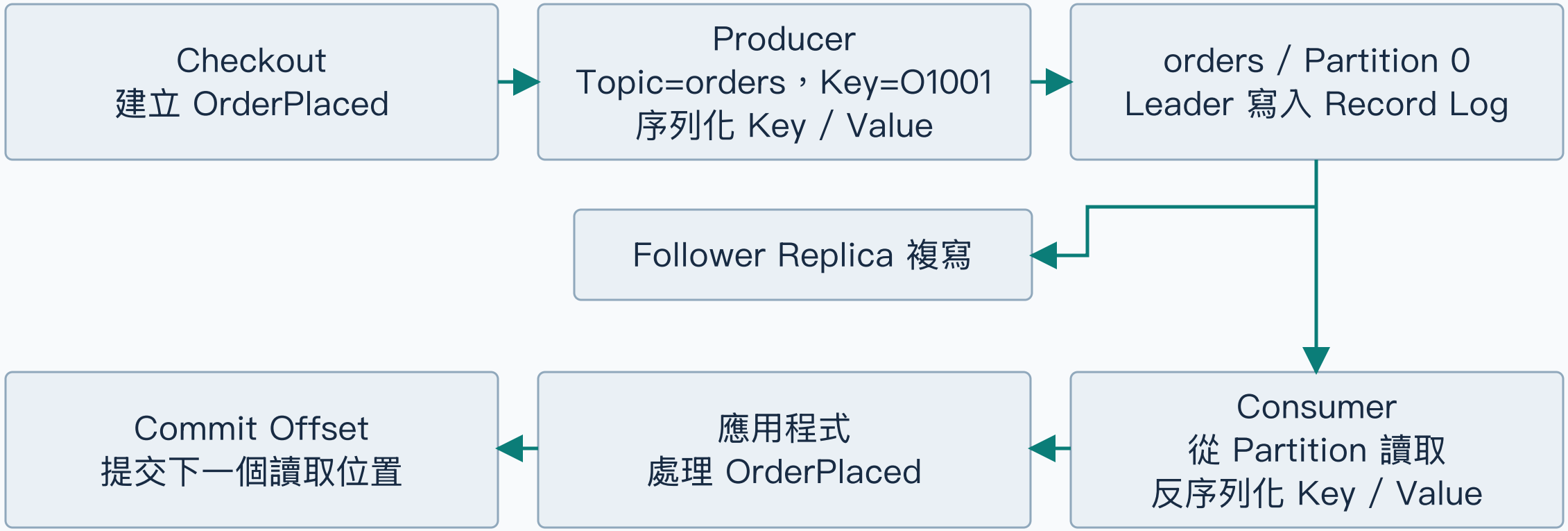
Exactly-once 讓 Kafka 交易範圍內的處理結果只生效一次，不代表程式只執行一次。

外部寄信、付款 API 不會自動包含在 Kafka Transaction 內。

七、重新複習一筆 Event 會怎麼被處理

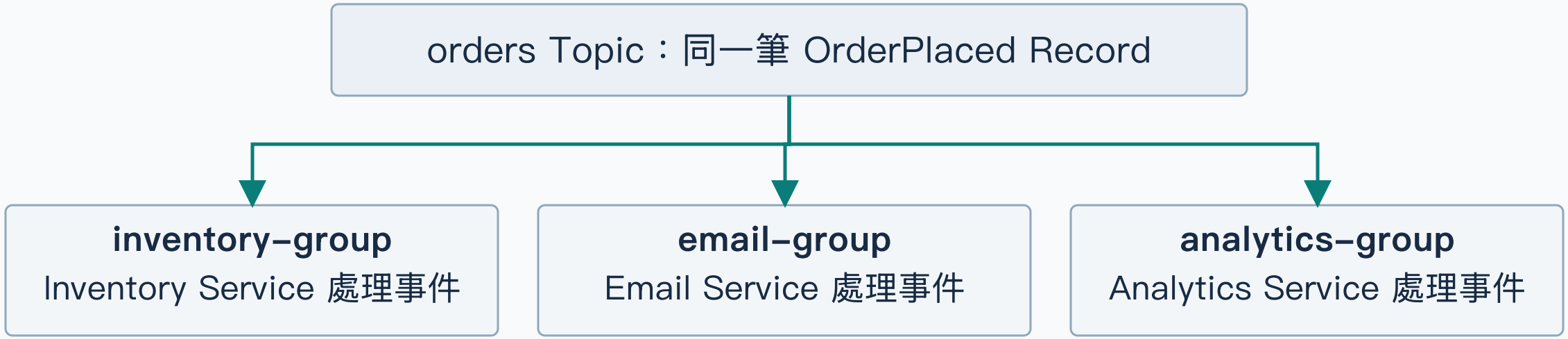
把所有名詞放回同一筆 OrderPlaced，重新走一次完整流程。

O1001 的完整旅程



Kafka 保存資料與進度，應用程式負責業務結果與重複執行的處理

同一筆 Event，供不同 Group 各自處理



各 Group 分別讀取同一批 Record，維護自己的進度

Kafka 保存資料與進度，業務是否成功與是否重複仍需要服務設計

一套理解 Kafka 的心智模型

遇到新的 Kafka Concept，先問：

1. **在哪裡？** 它位於 Event Flow 的哪個環節？
2. **為什麼？** 它想解決什麼問題？
3. **有什麼限制？** 它能保證什麼，又不能保證什麼？

把技術名詞放回 Event Flow，理解問題、機制與限制，而不只是背誦定義。

問題與概念：寫入、位置與讀取

實際問題	Kafka Concept
誰把 OrderPlaced 寫進 Kafka？	Producer
哪個 Server 儲存資料並提供讀寫？	Broker
訂單事件要寫入哪一類資料？	Topic
資料如何分散保存與平行讀取？	Partition
同一張訂單的事件如何依序讀取？	Key + Partition
一筆 Record 在 Partition 的哪個位置？	Offset
同一服務的多個 Consumer 怎麼分工？	Consumer Group

問題與概念：進度、保存與可靠性

實際問題	Kafka Concept
Group 故障後從哪個位置恢復？	Committed Offset
Group 的進度距離 Partition 尾端多遠？	Consumer Lag
修正統計後如何重算歷史資料？	Replay + Retention
Broker 故障，如何保存資料並接手？	Replication + ISR + Leader Election
誰管理叢集？管理者故障怎麼辦？	Controller + KRaft
寫入逾時，如何重試並等待確認？	Retry / acks
寫入重試如何避免新增重複 Record？	Producer Idempotence

延伸閱讀

- **Kafka Streams**：Stream Processing、Aggregation、Windowing
- **Kafka Connect**：Kafka 與 Database、外部系統的資料整合
- **KRaft**：Raft Consensus、Controller Quorum、Metadata Replication
- **Share Group**：Record-level Acknowledgement、工作分配模式
- **Reliability & Operations**：Consumer Lag 監控、Transactions、故障復原